

论文 Review: Optimal Resizable Arrays

万字 (241880496)

2026年7月11日

论文标题	Optimal Resizable Arrays
作者	Robert E. Tarjan, Uri Zwick
发表信息	SOSA 2023 (Symposium on Simplicity in Algorithms), 2023
Review 作者	万字 (241880496)
日期	2026-07-11

目录

1	引言	3
2	基础知识	3
3	这篇文章讲了什么?	3
4	这篇文章的分层	4
5	背景和铺垫	4
5.1	问题定义	4
5.2	对前人工作的引用和论述	5
5.2.1	Two basic data structure	5
5.2.2	The data structure of Sitarski	5
5.2.3	The data structures of Brodnik et al.	6
5.3	The lower bound of Brodnik et al. and its extension	6
5.4	新方法的提出	7
5.4.1	一个简单的($O(N^{\frac{1}{3}})$, $O(N^{\frac{2}{3}})$)的实现	7
5.4.2	一个($O(rN^{\frac{1}{r}})$, $O(N^{1-\frac{1}{r}})$)实现的优化	8
5.4.3	访问加速	10
5.4.4	进一步优化: 去除r这个小瑕疵	10
5.5	为何无法更牛逼	11
5.5.1	增长游戏	11
5.5.2	确实无法更牛逼了	11

6 分工说明	11
7 参考文献	12

1 引言

简要介绍本文 Review 的背景、目的和结构安排。

2 基础知识

O 代表上界, Ω 代表下界.

均摊分析是用来分析某一特定操作的开销的分析方法, 比如分析数组插入元素带来的开销。

均摊分析有聚合分析、记账分析、势能分析三种。

聚合分析就是先把所有操作的开销加起来然后求平均。

记账分析是一种简化版的势能分析, 假设某一动作自身消耗为 n , 算的时候假设是 $m+n$, 即留 m 给别人, 最后看 m 是多还是少还是刚好, 调整 m 即可

势能分析类似于中学物理学的势能和动能, 就是说在考虑一个操作的时候不能只看它自己的开销还要关注它对之后操作的开销的影响。

举个例子, 假设一个vector采用超容则乘2复制的方法。那一个超容的插入的开销是 $O(n)$, 但它同时也便利了之后几次插入操作, 因为它们不必再重复扩容和复制的过程了。

这就好比小球沿着非光滑曲面冲上高处, 消耗动能产生摩擦热同时储存重力势能。

堆栈操作是一个很好的均摊分析的例子。

假设一个栈 s 支持三种操作, 开销如下:

push: $O(1)$; pop: $O(1)$; pop_k: $O(\min(s.size(), k))$;

聚合分析:

n 次push: $O(n)$

所有pop: $O(n)$

平均: $O(1)$

记账分析:

每一次push设代价为2, 留1

正好与pop抵消, 故为 $O(1)$

势能分析:

设一个势能函数 $\Phi(s)=s.size()$, 即将元素的数目看作资源

则push资源增加, 开销为2

pop资源减少, 开销为0

故为 $O(1)$

3 这篇文章讲了什么?

一共就讲了两个重点: 一是在给定的条件下给出了一种数组的实现, 使得访问代价为 $O(1)$ 且空间为 $N + O(N^{1/r})$, 这是他们的核心结论, 比之前的同方向的研究更好。

另一个是给定一个下界，说明你不可能做到一个数组的实现：在保持空间 $N + O(N^{1/r})$ 的同时还指望 grow 的时间小于 $\Omega(r)$ ，也就是让其他人别再试图往那个方向钻了。

4 这篇文章的分层

- 第一节：Introduction
- 第二节：问题定义
- 第三节：讲述前人的工作
- 第四节：关于一个有用的 lower bound
- 第五节：描述 $r = 3$ 时的 resize 和 store 开销
- 第六节：扩展至任意 $r \geq 2$ 的情况
- 第七节：对第六节中的方法的一个小瑕疵进行优化
- 第八节：提出抽象增长游戏（一个抽象的数学模型）
- 第九节：用抽象增长游戏证明在空间为 $N + O(N^{1/r})$ 时他们的实现是最优的

接下来的 review 我会按照原文的结构分成四个部分：

背景和铺垫：1-4

提出新方法：5-7

证明无法更牛逼：8-9

我自己的一些思考

5 背景和铺垫

5.1 问题定义

这篇文章讨论的问题不包括在 array 中间加或改元素的情况。

他们还假设了他们的系统里可以使用类似于 free 和 malloc 这样的工具来分配和释放任意位置和大小空间且这种操作对每个单位的代价是常数级的。也就是说，他们不管内存管理系统的实现，默认它已经足够好了。然后他们介绍了一种叫做 block 的东西，他们令一个可变数组由若干定长数组的组合来实现的。这篇文章讨论的所有数据结构本质上都是 blocks。这是一个值得注意的地方，因为所有主流编程语言的可变数组都是由连续内存空间来实现的而不是用所谓的 block。实际上我之前从来没听过 block 这种设计。block 一共可分为三种：数据 block：直接存目标数据，索引 block：存指向其它 block 的指针和辅助 (auxiliary) block：存一些别的。这有点类似于操作系统中的分页内存管理机制中的页表和数据块和辅助位。

而这种分类是何意味呢？这主要是为了引出他们自己给出的一个丈量即：Definition 2.1($(s(N), t(N))$ -implementation). Let $s(N), t(N)$ be two non-decreasing functions. A resizable array data structure is said to be an $(s(N), t(N))$ -implementation if it uses at most $N + s(N)$ space to store an array of size N , and at most $N + t(N)$ space during a grow or shrink operation on an array of size N . 这个定义几乎贯穿全文也是本文最大的创新。即将一个可变数组的空间消耗分两种情况来看一种是增或减时的开销以及其它时候的开销。也就是他们敏锐的捕捉到了一个可变数组的开销不是一成不变的，这里面有可以操作的空间。而之前的所有研究者都没有抓住这一点。

5.2 对前人工作的引用和论述

5.2.1 Two basic data structure

一共讲了两种基本的数据结构，第一种是最蠢的，用一个 N 空间的定长数组来模拟一个可变数组，每一次删除或增加元素都要重新分配空间然后完成复制。这几乎没有意义，作者写它估计是为了水字数。第二种是用的最广的即 geometric expansion/shrinking 结构，就是我们最熟悉的C++的vector的实现。均摊分析算下来grow 的代价为 $(1+1/a)$, shrink的代价为 $(1/a)$ 。这个结构已经不错了，也非常practical。

5.2.2 The data structure of Sitarski

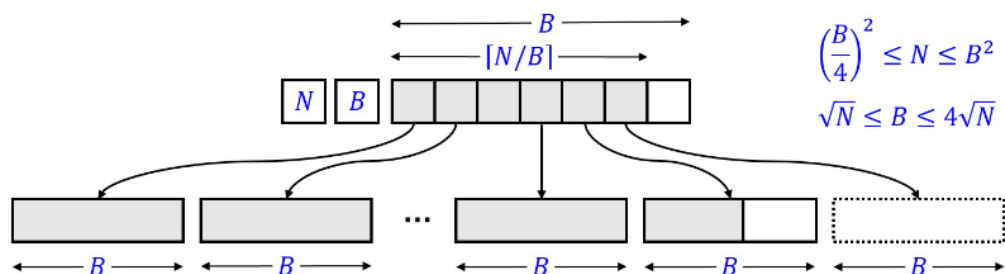


Figure 1: Sitarski's HAT data structure.

Sitarski的实现名字叫hashed array tree(HAT)。名字很高级但实际上既没有哈希也没有树。它本质上就是数组套数组，还是页表那一套。它就是把一个数组切成小块每一个block大小为 B ，然后再来一个索引块，也为 B ，所有块大小都是 B 。找元素的时候就查两次数组就好了。它的优势在于每次超容的时候只要加一个索引指向一直被维护的空数据block即可。但其实它复杂度和geometric差不多。总的来说这还是一种很简单的很容易想到的实现。下一个实现就是它的变形。

5.2.3 The data structures of Brodник et al.

这个比较有意思，这个实现的美妙之处是 no rebuild is necessary and data items do not need moving。它的实现思路和上一个类似。都是先查指针再插数据块。但不同之处在于每个数据块大小不一且呈现递增趋势。之前Sitariski的做法的缺点在于当N越来越大，B由于要满足单次查询 $O(1)$ 的条件而不得不扩大，这一扩大，就要重建和搬元素。但是Brodnik的做法却不同，由于B自动增大，假设按1,2,3,4,.....n这个顺序来,最后的B和N的关系刚好是 $B = \sqrt{2N}$ 。但复杂度和前两种差不多，依旧是 $N + O(\sqrt{N})$ 。作者之后拿这个算法开刀来展开他的理论

5.3 The lower bound of Brodник et al. and its extension

前面已经引出了Brodnik的实现，接下来提出了三个结论如下：

定理 5.1. *Any data structure for maintaining a resizable array must at certain times use $N + \Omega(\sqrt{N})$ space, where N is the current length of the array, even if only grow and access operations are performed.*

定理 5.2. *Any $(s(N), t(N))$ -implementation of resizable arrays must have $s(N) \cdot t(N) \geq N$, even if only grow and access operations are supported.*

定理 5.3. *For any integer $r \geq 1$, any data structure that uses only $N + O(N^{1/r})$ space for storing a resizable array of size N must occasionally use $N + \Omega(N^{1-1/r})$ space during a grow operation, even if only grow and access operations are performed.*

我来展开说明。

首先，需要注意，这里所有的结论都是针对Brodnik的实现。有一个关键点在于这种实现下,Block的大小是递增的。

1. Theorem 4.1 说明：这个定理是说一个数组在扩容的时候，总有那么一个时刻，假设这个时刻它的元素的数目为 N ，那么此时它所需要的空间为 $N + \Omega(\sqrt{N})$ 。证明也很简单，显然随着N：数组元素的增加，k:索引Block的数量的增加是跳跃式的，即一个数组可能在两个时刻k相同但是Block的数目差距很大。显然所有的数据块都是必须的，这一部分开销为N，而索引块的容量比较复杂，因为是跳跃性的增加的。所以分类讨论。当 $k \geq \sqrt{N}$ 时，空间消耗为 $N + \Omega(\sqrt{N})$ 。但这只是极少数情况时k的取值，实际上k常常小于 $\sqrt{N}$ 因为现有的索引块指向的数据块不被装满才是常态。当k小于 $\sqrt{N}$ 时,平均下来每个数据块的大小必然大于 $\sqrt{N}$ ，根据鸽巢原理，必然存在一个数据块，它的大小大于 $\sqrt{N}$ 。回看这个大的数据块刚刚被分配的时候也就是它里面还是空的时候，假设此时数据块的总容量为 N' 显然此时的空间消耗为 $N' + \Omega(\sqrt{N'})$ （这里我们甚至没考虑指针的开销）。证明完毕。

2. Theorem 4.2 说明：这个的证明其实和上面的4.1类似。首先重申一遍 $s(N)$ 和 $t(N)$ 的定义。 $s(N)$ 表示在任何时候存储一个大小为 N 的可变数组所需的最多空间， $t(N)$ 表示在增长或缩小操作期间使用的最多空间。我们依旧只讨论除存数据的空间以外的开销。

只要关注索引块和数据块和空白块的相对变化就好了。当空白块为空的时候所有的额外开销都用于储存索引，此时索引的数量就是 $s(N)$ (有个前提，索引项的大小是一个字，而以上我们所有的讨论都是以字为单位，忘了说了哈哈)。那么根据鸽巢原理，必然存在一个数据块，它的大小 $\geq \frac{N}{s(N)}$ 。那么我们再回到这个大块被创建的时候还是空的时候，设此时的元素数目为 N' ，显然此时的开销为 $N' + \frac{N}{s(N)} + \text{else}$ 。所以 $t(N') \geq \frac{N}{s(N)}$ ，回到元素数目为 N 的时候，由于 $t(N) \geq t(N')$ ，所以 $t(N) \geq \frac{N}{s(N)}$ 。所以 $s(N) \cdot t(N) \geq N$ 。证毕。

注意，这个约束非常重要，因为无论层数 r 为多少，这个定理都成立，之后会反复用到。

3. Corollary 4.3 说明：这就是从上一个结论导出来的必然结果， $\frac{1}{r}$ 只是一个人为选定的参数，不用纠结。

5.4 新方法的提出

5.4.1 一个简单的($O(N^{\frac{1}{3}}), O(N^{\frac{2}{3}})$)的实现

这是一个承上其下的部分，通过之前的介绍，作者提出了一个简单的实现，满足空间为 $N + O(N^{1/3})$ ，增长操作的开销为 $O(N^{2/3})$ 。

这个实现的思路是这样的：

既然要实现 $s(N)$ 为 $O(N^{1/3})$ ，那么索引块的数量就必须是 $O(N^{1/3})$ ，那么每个数据块的大小就必须是 $O(N^{2/3})$ 。我们考虑沿用之前提到的HAT的实现思路，即索引块指向数据块的一层查找。但是问题很快出现了，依旧考虑数据块刚被分出来的时候，因为它的大小为 $N^{\frac{2}{3}}$ 。显然此时光是这一块的浪费了的开销为 $N^{\frac{2}{3}}$ ，这就不满足 $t(N)$ 为 $O(N^{1/3})$ 的要求了。但是要满足($O(N^{\frac{1}{3}}), O(N^{\frac{2}{3}})$)这个实现，索引块的数量又不能太大。直接一根筋变两头堵了。

作者的解决方案是引入辅助块或者说缓冲块的方法。

也就是说，大体上还是延续之前的HAT的实现思路，但是对于最后分配的那一个内存块，我们不直接把它分配为一个数据块，而是先分配为一个辅助块（它能储存的元素数目上界为 $N^{\frac{2}{3}}$ ），等到这个辅助块被填满了之后再把它转化为一个数据块。而这个辅助块用经典的 $r=2$ 的HAT来实现。用页表来类比（即B的大小为 $N^{\frac{1}{3}}$ ），就是一个混合页表。

因为不直接存储一个大的数据块而是内部维护一个容量缓慢增大的辅助块，所以在这个辅助块被填满之前，空间开销是非常小的。额外的开销控制在 $\Omega(N^{1/3})$ 。

至此，我们就达成了我们的目标。

5.4.2 一个 $(O(rN^{1/r}), O(N^{1-1/r}))$ 实现的优化

从前一小节中我们已经能看出，只要 $r \geq 2$ ，我们就能实现一个 $(O(N^{1/r}), O(N^{1-1/r}))$ 的实现。就好像多级页表一样，我们完全可以数组里面套数组来实现空间的高效利用。但这种递归的实现方式太过复杂了，假如说 $r=5$ ，那层数就太多了，作者提出了一个优化的实现。它在空间复杂度上等价（之后会说明这一点）但它是扁平的，易于理解的。简单来说，在递归的实现方式下不是存在一个块的大小和容量的平方关系吗，那就直接分配块的时候就按照这个来就好了。这样就不用查来查去了（怎么有种伙伴系统的感觉，只不过伙伴系统里面是2的幂这里是B的幂）。

具体实现见下图：

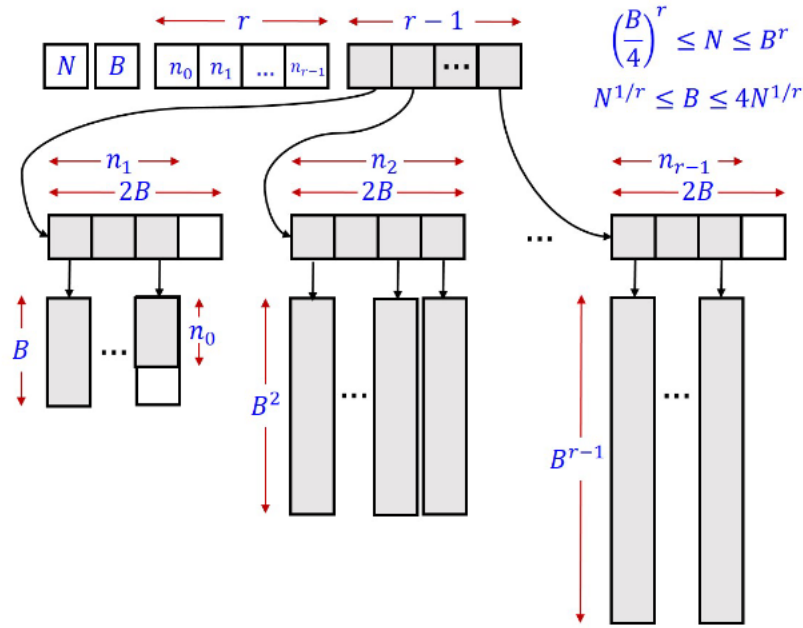


Figure 4: An $(O(rN^{1/r}), O(N^{1-1/r}))$ -implementation, for every $r \geq 2$.

这种扁平化的实现方式的好处是它的层数只有两层，索引块和数据块故你想要访问任何一个数据块的时间开销都是 $O(r)$ 。并且它的空间复杂度和之前的递归实现方式是等价的。

对这种数据结构进行元素的删或增是非常容易的，只需要记录当前有哪些大小的块和各个大小的块的数目就好了。如果操作是增，且目前没有空位则先检查合并可能，倘若元素块已经可以向上合成了（用一个位图来表示就是 2B-12B-12B-12B-12B-1，只要最后一位是2B-1即可，至于为什么不是B-1，这是一个小巧思），那这种情况直接向上合成到不能合成再分配小块即可。否则直接分配就好了。删的操作就是反过来，只是将检测是否可合并改为是否需要拆分。

然后下面我给出这种思想的伪代码实现(核心意思就是我这里说的):


```

Grow(a):
  if  $N = B^r$  :
    | Rebuild( $2B$ )
  else if  $n_1 = 2B$  and  $n_0 = B$  :
    | Combine-Blocks()
  else if  $n_1 = 0$  or  $n_0 = B$  :
    |  $A[1][n_1] \leftarrow Allocate(B)$ 
    |  $n_1 \leftarrow n_1 + 1$ 
    |  $n_0 \leftarrow 0$ 
   $A[1][n_1 - 1][n_0] \leftarrow a$ 
   $n_0 \leftarrow n_0 + 1$ 
   $N \leftarrow N + 1$ 

Combine-Blocks():
   $k \leftarrow \min\{i \in [r - 1] \mid n_i < 2B\}$ 
  if  $k = \infty$  : error
  for  $i \leftarrow k - 1$  downto 1 :
     $A[i + 1][n_{i+1}] \leftarrow Allocate(B^{i+1})$ 
    for  $j \leftarrow 0$  to  $B - 1$  :
       $Copy(A[i][j], 0, A[i + 1][n_{i+1}], jB^i, B^i)$ 
       $Deallocate(A[i][j])$ 
       $A[i][j] \leftarrow A[i][j + B]$  // Shift indices.
     $n_i \leftarrow B$ 
     $n_{i+1} \leftarrow n_{i+1} + 1$ 

Shrink():
  if  $N = (B/4)^r$  :
    | Rebuild( $B/2$ )
  else if  $n_1 = 0$  :
    | Split-Blocks()
   $n_0 \leftarrow n_0 - 1$ 
   $N \leftarrow N - 1$ 
  if  $n_0 = 0$  :
     $Deallocate(A[1][n_1 - 1])$ 
     $n_0 \leftarrow B$ 
     $n_1 \leftarrow n_1 - 1$ 

Split-Blocks():
   $k \leftarrow \min\{i \in [r - 1] \mid n_i > 0\}$ 
  if  $k = \infty$  : error
  for  $i \leftarrow k - 1$  downto 1 :
     $n_{i+1} \leftarrow n_{i+1} - 1$ 
    for  $j \leftarrow 0$  to  $B - 1$  :
       $A[i][j] \leftarrow Allocate(B^i)$ 
       $Copy(A[i + 1][n_{i+1}], jB^i, A[i][j], 0, B^i)$ 
     $Deallocate(A[i + 1][n_{i+1}])$ 

```

接下来看看Grow和Shrink操

作的时间复杂度的均摊分析。因为这是一个涉及到空间扩充的操作，所以直接用记账分析就好了。这里原文讲了三页纸但其实很简单。

首先我们需要一点直觉即这种数据结构的开销必然和它的层数 r 挂钩，因为你 r 越多，分出的块就越多，需要转表的次数也就越多(这一点在之前提到的递归的实现里反而体现的更明显，这里扁平化了其实不太好看)。

所以我们可以假设每次操作的开销为 $O(r)$ ，但这个账显然不是平均分布的，那些越小的子块变动的越频繁，因为它们容量小，它们更容易发生扩容和合并给它们记 r 。而那些最大的块里的元素，它们从被创建开始就没动过。所以直接给它们记0，设层数为 i ，发的账为 $r-i$ 。我们接下来只讨论合并的操作的开销就好，因为单纯的增加元素开销就是 $O(1)$ 而且只用做一次。而没一次下层的元素往上移动就是用上层的空间中复制下层的元素来实现的，所以每一个元素向上跃迁一次，开销就是 $O(1)$

如果一个块的大小为 B^k 也就是说它是由大小为 B 、 B^2 、 B^3 、...、 B^{k-1} 的块合并而成的，那么它的开销就是 $B \times (B + B^2 + B^3 + \dots + B^{k-1}) \leq 2B^k$

而之前我们给每一个元素发了 $(r-i)$ 的账， i 是它所在的块的层数。第 i 层的块的大小为 B^i ，所以每一个元素的账为 $r-i$ ，而每一个块的大小为 B^i ，所以每一个块的账为 $B^i \times (r-i)$ 而从第 i 层到 k 层消耗的账为 $B^i \times (i - k)$

每一层的消耗的账为 $B^i \times (r-i)$ ，而从第 i 层到 k 层消耗的账为 $B^i \times (i - k)$ 刚好相等。(也可以加起来求平均，是一个等差数列乘以等比数列的形式，可以求和)。所以每一个元素的账都足够支付它的开销。

所以Grow的均摊开销为 $O(r)$ 。Shrink同理。

同时作者还有一个小巧思，就是冗余加速。比方说 $1999+1$ 这个运算，正常来说要进位3次。但作者可以允许冗余，即只有一位等于20的时候才进位，这样就极大的减少了

进位的次数，也就减少了内存块复制的次数。而为了进一步减少级联的进位操作还规定一次运算只能发生一次进位，倘若本次运算没有触发抵达2B则扫描高位是否溢出，溢出则处理。不难看出在这种情况下每一次加法最多带来两次位的变化，所以开销直接变成 $O(1)$ 。

5.4.3 访问加速

这里说白了就是经典的位运算加速访问。之前的老访问方式是用多级页表的方式去一一次的查表，显然这种方式对页表这种地址缺乏规律的情况是最好的但目前这种分块有一个很重要的东西是块的大小是B的幂。直接把要找的元素的索引和目前元素的总数转为B进制数。假如目标元素总数为N此时的B为8，转为 N_B 后为(1,2,3,4,5,6,7)而索引转换后为(1,2,3,3,1,1,1)，那么就应该去第四位找那个元素。但这种情况成立不代表永远成立，因为我们的表示是冗余的而之前的表示方法都是传统的转化为B进制，这种方法无法正确反映每一个size的内存块的数量。所以要采用补救措施。

比方说 $B=5$ 时 N_B 为(1,0,3,1)(位数左低右高)， S_B 为(0,3,2,1)。从高到低比发现第二位首次出现小于关系，如果数据的实际存储就是按照标准的B进制来，那这里就可以结束了，但实际存储为冗余存储因为要保证grow的速度（见前文）。这里发现是落在2号块里，最理想的情况是索引刚好位于1号块的和与二号块和一号块的和之间，这样就能一步到位因为此时有没有冗余都一样，直接就能判断去哪里找元素。但此时一号块的和只有125，不满足，所以要打补丁。先看上帝视角， $N_B=(6,9,6,0)$ ， $S_B=(5,7,6,0)$ ，显然在第1号位。但是这个转换在计算机里太复杂了，不可能每次打补丁都要这么做。所以要做差。注意这里有一个前提就是每一位是否存在冗余，数据结构都是维护了的。所以因为差是15而二号位的上一位也就是一号位没有发生冗余故容量为20，装下了，所以在1号块里。

综上，通过这种方式，成功实现了访问加速，现在的访问速度直接到了 $O(1)$ 。

5.4.4 进一步空间优化：去除r这个小瑕疵

之前给出了一个 $(O(rN^{\frac{1}{r}}), O(N^{1-\frac{1}{r}}))$ 的实现都是基于r为一个固定的常数的。在这种情况下，我们已经论证了它是最优的。但是实际上，r可以是一个随着N变化而变化的值。直觉就是随着N的增大，原有的层数太少了，因为当B给定的情况下如果r给定，N就定了，所以如果当前的r容不下N，那N就要变化，所以显然存在一个N到r的映射。因为这是一个等比数列和的关系，所以 $r=r(N)=O(\log N)$ 当然这只是便于理解，不然这个r一直出现会把人搞蒙。接下来我们在这个语境下讨论将存储时所需的空间进行优化。

从静态和动态入手。首先我们要建立一种直觉，一个数据结构要被存储，它大多数时候都是静态的，此时它的空间消耗为 $N + s(N)$ ，但是当你需要进行增和删的时候，就需要开辟一个临时空间并进行元素的复制。显然，这个临时空间越大，需要的 $t(N)$ 也就越大。但是这个数组大部分时候都是静态的，只有当分配了新的块的时候它才需要考虑 $t(N)$ 的开销。前文提到过 $s(N) \cdot t(N) \geq N$ ，说明它们之间存在某种此消彼长的关

系，那么我们是否可以以 $s(N)$ 小幅上涨为代价让 $t(N)$ 降下来？这是可行的，实际上可以将 $(s(N), O(N))$ 的实现转化为 $(s(N) + O(\frac{N}{t(N)}), s(N) + O(t(N)))$ 的实现，只要内存分配方式是一次分配一次到位并清空旧空间。（这个条件很常见，不用在意）接下来我来证明它：

这不是严格的那种数学证明实际上也不需要这么做，因为这里本质上是在讨论内存的分配方式而没有改变数据结构的底层逻辑，不同的分配方式带来不同的效用，根据前文提到的所有约束我们可以看出这个实现是正确的因为它满足 $s(N) \cdot t(N) \geq N$ ，也就是说确实存在这种实现。我们只要找出这种数学上的模型的具体实例就能证明这是可行的了。还记得之前我们提到过对于一个 $(O(N^{\frac{1}{3}}), O(N^{\frac{2}{3}}))$ 的讨论时为了让 $s(N)$ 缩小到 $O(N^{\frac{1}{3}})$ 时做的工作吗？这里也是差不多的。我们发现，当一个resized array增长时，临时空间相对于当前的 N 的最大的时候就是当临时空间刚刚被分配的时候。此时，旧的待复制的内存块还在，而新的待填充的内存块也在，这个瞬间，对空间的压力是最大的。所以我们要想方设法解决这个问题，即旧的和新的同时存在的问题，只要让新的空间被分配的同时复制一部分旧的元素同时释放掉一部分旧的空间就行了。也就是不要一次性搬家，只要等新家的一个房间装修好了就可以从旧的家开始搬东西了。说白了就是新的块的粒度细一点，浪费的空间小一点。接下来我们假设某一时刻分配了一个块，它的大小大于 $O(N^{1-\frac{1}{r}})$ 即大于 $t(N)$ 。（小于它的空间不用讨论，那可以接受）那就不要一次性分配这个块，而是分配 $\frac{N}{t(N)}$ 个块按照 $B, B^2, B^3, \dots$ 来分配空间就好了（ B 在此时的大小还是满足 $N^{\frac{1}{r}} dB < 4N^{\frac{1}{r}}$ ），这就和之前提到的那个实现如出一辙。现在我们知道这种实现的过程峰值不可能大于 $s(N) + O(t(N))$ ，因为一共就一个没装满的空间且它不可能大于 $t(N)$ 。那为了维护这 $\frac{N}{t(N)}$ 个块，就需要这么多个指针，所以永久存储就是 $s(N) + O(\frac{N}{t(N)})$ 这么多。所以对于 $(O(rN^{\frac{1}{r}}), O(N^{1-\frac{1}{r}}))$ 这个实现，我们带入进去就能把它变成 $(O(rN^{\frac{1}{r}}), O(rN^{\frac{1}{r}} + N^{1-\frac{1}{r}}))$ 这个式子很丑，引入一个不等式 $r \leq \frac{\log(N)}{2\log(\log N)}$ 这是一个对 r 的取值的约束。然后就得到了一个漂亮的式子： $(O(rN^{\frac{1}{r}}), O(N^{1-\frac{1}{r}}))$

这个实现的访问开销均摊下来和前一个一样。都是 $\Omega(r)$ 。

至此，这篇文章的第一个要点就将完了，就是反复的在临时和恒定里做文章。

5.5 为何无法更牛逼

5.5.1 增长游戏

增长游戏的篇幅很长，有6页都在讲这个玩意儿。但说白了，他就是对现有的数据结构做一个简化，之间不是贴了个伪代码吗？这个增长游戏就是把其它操作的开销删掉，只留下元素拷贝的开销。因为作者想证明的是如果你只允许 $N + cN^{1/r}$ 的永久额外空间（即 $kHcN^{(1/r)}$ ），那么均摊代价至少是 $\Omega(r)$ 。

5.5.2 确实无法更牛逼了

6 分工说明

本项目所有工作由同一人独立完成。包括：

- 论文选题与精读
- 文献检索与综述
- Review 正文撰写
- 个人反思写作
- 项目管理与版本控制

AI 工具（GitHub Copilot / ChatGPT 等）的使用记录见 `records/ai-usage.md`。

7 参考文献

详见 `references/bibliography.md`。